FFmpeg and Whisper Video Processing Architecture

https://gemini.google.com/app/abfa417bbcba5038

User prompt: analyse this pdf thoroghly and make sure the application must process video through wisper and ffpeg and audio of the video

Response: Based on the **Multimodal Study Companion — MVP System Architecture** diagram provided, here is a thorough analysis of the architecture, including explicit verification of the video and audio ingestion pipeline via FFmpeg and Whisper.

Architecture Overview & Core Modules

1. Student Experience (`Vercel`)

- **Tech Stack**: Next.js + TypeScript + Tailwind CSS.
- **Key Features**: Upload functionality, tutor chat, quizzes, and mastery dashboard.
- **Authentication**: Supabase Auth (Email / Google) with JWT session handling.
- **Media & Documents**: Integrated viewer using `PDF.js` for documents and `video.js` for video content. Slides are rendered as PDFs, citations jump directly to specific pages/slides or seek exact timestamps in original videos.
- **Real-time Streaming**: Utilizes Server-Sent Events (SSE) to stream tutor answers and upload progress.

2. Application API (`Render / Docker`)

- **Tech Stack**: FastAPI + Pydantic + SQLAlchemy.
- **Security & Controls**: JWT validation, course ownership checks, signed URLs, REST endpoints, and request rate limits.
- **Core Components**:
  - **Application Orchestrator**: Handles chat, assessments, profile intake, diagnostic quizzes, and persists learning events.
  - **Celery + Redis**: Manages background ingestion jobs, retries, exam handling, and idempotent task execution.
- **Deployment**: Single Python codebase deployed separately as API and Celery worker instances (no complex Kubernetes required for MVP).

3. Ingestion Worker (`Render / Docker`)

- **Automatic Text Extraction**:

    - **PDFs**: Handled via `PyMuPDF` + `Tesseract` OCR.
    - **PPTX**: Converted using `LibreOffice` → PDF.
    - **Videos & Audio Processing**:
        - **`FFmpeg`**: Processes raw video files to extract/manipulate audio tracks and frames.
        - **`Whisper`**: Transcribes the extracted audio into text while preserving timestamps and page/slide/timecodes.
- **Visual Extraction**: Uses `GPT-4.1 mini vision` to process keyframes, figures, diagrams, and generate descriptions linked to originals.
- **Normalization & Chunking**:
    - Uses `GPT-4.1 mini` for topic, concept, and prerequisite extraction.
    - Uses `text-embedding-3-small` to produce chunk and figure embeddings.
    - Metadata stores `source_id`, `page`, `slide`, `t_start/t_end`, and `topic_ids`.
    - Versioned content hashes prevent redundant ingestions.

4. Course & Learner Data (`Supabase`)

- **Supabase Storage**: Private buckets holding original videos, PDFs, PPTX, rendered slides, keyframes, and extracted figures.
- **PostgreSQL + `pgvector`**:
    - **Vector & Excerpt Store**: Chunks, excerpts, vectors, source locators, topics/concepts, prerequisite edges, and course-scoped metadata.
    - **Learner Tables**: Profiles, topic mastery records, evidence events, question banks, exposure history, and attempt logs (enforced via Row-Level Security).

5. Tutoring & Adaptive Learning Modules

- **Hybrid Retrieval**: PostgreSQL Full-Text Search (FTS) + `pgvector` dense search with Reciprocal Rank Fusion (RRF) for top-k context retrieval.
- **Grounded Tutor**:
    - Driven by `GPT-4.1 mini` + retrieved evidence.
    - Validates citation IDs and excerpts; strictly refuses to answer without sufficient evidence (eliminates silent outside knowledge).
- **Assessment Pipeline**:

    - Validates MCQs, short answers, and numerical problems using `GPT-4.1 validator` and `SymPy`.

- **Learner Model**: Implements Bayesian Knowledge Tracing (Python BKT) for per-topic mastery and confidence tracking.

6. Model Gateway & Quality Assurance

- **OpenAI SDK / Typed Model Gateway**: Manages model calls with rate limits, timeouts, token budgets, and independent answer-key validation.
- **Evaluation & Benchmarking**:
    - **RAGAS + pytest + GitHub Actions**: Benchmarks faithfulness, answer relevancy, context precision, and context recall.
    - **Student Simulation + Langfuse**: Runs multi-session test runs to evaluate mastery change, latency, costs, and traces.

Ingestion Pipeline Verification (FFmpeg + Whisper)

Target Media	Tool / Pipeline	Function in System Architecture
Video Files	`FFmpeg`	Demuxes/extracts audio tracks from video files and extracts keyframes for visual processing.
Audio Processing	`Whisper`	Transcribes extracted audio stream to text, retaining precise temporal timecodes (`t_start`, `t_end`) for precise video seeking in `video.js`.
